

08 - verifica di sistemi reattivi

Fondamenti Logici dell'Informatica - Modulo 2

Corso di Laurea Magistrale in Informatica – A.A. 2024-2025

Università di Bologna

limiti del testing

Program testing can be used to show the presence of bugs, but never to show their absence!
Dijkstra [1970]

Finitezza dei test

- il numero di test che possono essere effettuati è sempre finito

Contesti in cui il testing è inefficace

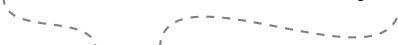
- multithreading, multitasking, parallelismo, distribuzione rendono l'esecuzione dei programmi imprevedibile
- è difficile svolgere dei test esaustivi in modo sistematico

verifica di sistemi critici

- è necessario adottare tecniche di verifica **più efficaci del testing**
- soprattutto se il sistema malfunzionante può causare gravi danni
 - operazioni finanziarie
 - controllo di mezzi di trasporto (treni, metropolitane automatizzate, aerei, droni, ...)
 - dispositivi medici
 - impianti industriali e di produzione dell'energia (dighe, centrali elettriche, ...).
 - ...

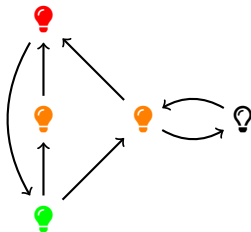
Model checking

- uso di logiche per specificare il comportamento atteso di un sistema
- tool verifica se un **modello** del sistema soddisfa la **specifica**


$$\mathcal{M} \models F$$

esempio: semaforo stradale

Modello



Specifica (esempi)

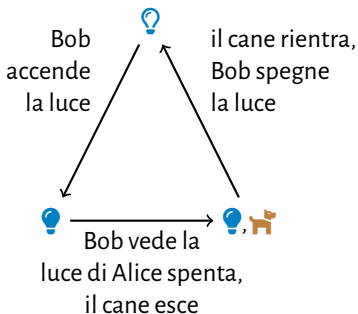
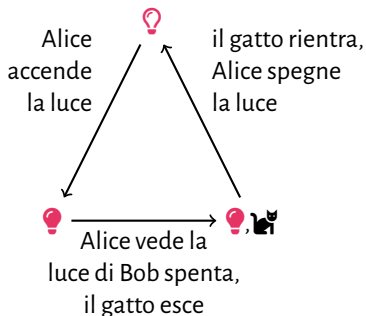
- non sono **mai** accese due luci diverse contemporaneamente
- se una luce è accesa in uno stato, è spenta in quello **successivo**
- se il semaforo è rosso, **prima o poi** diventa verde
- se il semaforo è verde, **prima o poi** diventa rosso

esempio: un problema di mutua esclusione

- Alice e Bob abitano in case distinte ma con un **cortile in comune**
- Alice ha un gatto e Bob un cane, che **si detestano**
- Alice e Bob concordano un **protocollo** per l'uso del cortile

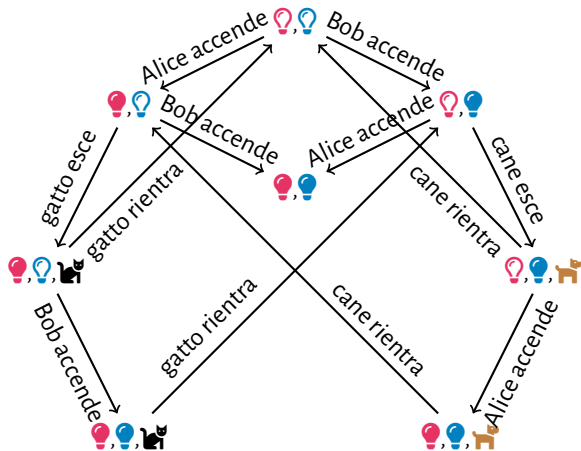
esempio: un problema di mutua esclusione

- Alice e Bob abitano in case distinte ma con un **cortile in comune**
- Alice ha un gatto e Bob un cane, che **si detestano**
- Alice e Bob concordano un **protocollo** per l'uso del cortile



Questo protocollo garantisce l'uso in mutua esclusione del cortile?

un modello per il protocollo Alice-Bob



Semantica “interleaving” della concorrenza

- azioni **indipendenti** vengono eseguite in sequenza, ma in **ordine arbitrario**

proprietà desiderate del sistema (esempi)

Safety = non succede **mai** nulla di male

(EX) *dallo stato iniziale* non è possibile raggiungere uno stato in cui sia il gatto di Alice che il cane di Bob sono in cortile

proprietà desiderate del sistema (esempi)

Safety = non succede **mai** nulla di male

(EX) *dallo stato iniziale* non è possibile raggiungere uno stato in cui sia il gatto di Alice che il cane di Bob sono in cortile

Progress = il sistema può **sempre** evolvere

(PA) Alice e Bob riescono *sempre* ad accendere la propria luce, se è spenta

(PS) Alice e Bob riescono *sempre* a spegnere la propria luce, se è accesa

proprietà desiderate del sistema (esempi)

Safety = non succede **mai** nulla di male

(EX) *dallo stato iniziale* non è possibile raggiungere uno stato in cui sia il gatto di Alice che il cane di Bob sono in cortile

Progress = il sistema può **sempre** evolvere

(PA) Alice e Bob riescono *sempre* ad accendere la propria luce, se è spenta

(PS) Alice e Bob riescono *sempre* a spegnere la propria luce, se è accesa

Liveness = **prima o poi** succede qualcosa di buono

(LA) se Alice accende la propria luce, prima o poi il gatto esce

(LB) se Bob accende la propria luce, prima o poi il cane esce

...si parla di **tempo**!

strutture di Kripke

Si fissa un insieme AP di proposizioni atomiche

- 💡 = “luce di Alice accesa”
- 🐱 = “il gatto è in cortile”
- ...

Una struttura di Kripke su AP è una quadrupla (S, s_0, R, L)

- S è un insieme di **stati**
- $s_0 \in S$ è lo **stato iniziale del sistema**
- $R \subseteq S \times S$ è la **relazione di transizione**, che supponiamo totale: per ogni $s \in S$ esiste $t \in S$ tale che $(s, t) \in R$
- $L : S \rightarrow \wp(AP)$ è una **funzione di etichettatura** che dice, per ogni stato, quali proposizioni atomiche sono vere in quello stato

Note

- spesso S deve essere *finito* per avere decidibilità del model checking
- scriveremo $s \rightarrow t$ per indicare che $(s, t) \in R$ se R è chiara dal contesto

struttura di Kripke per il sistema Alice-Bob

Proposizioni atomiche

- $AP \stackrel{\text{def}}{=} \{\text{💡}, \text{💡}, \text{🐱}, \text{🐱}\}$
- nota: $\text{💡} = \neg \text{💡}$

Stati

- $S = \{s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7\}$
- s_0 è lo stato iniziale

$$L(s_0) = \emptyset$$

$$L(s_2) = \{\text{💡}, \text{🐱}\}$$

$$L(s_4) = \{\text{💡}, \text{💡}\}$$

$$L(s_6) = \{\text{💡}, \text{🐱}\}$$

$$L(s_1) = \{\text{💡}\}$$

$$L(s_3) = \{\text{💡}, \text{💡}, \text{🐱}\}$$

$$L(s_5) = \{\text{💡}\}$$

$$L(s_7) = \{\text{💡}, \text{💡}, \text{🐱}\}$$

Relazione di transizione

- $R = \{(s_0, s_1), (s_0, s_5), (s_1, s_2), (s_1, s_4), \dots\}$
- in generale il sistema può essere non-deterministico
- per rendere totale R possiamo aggiungere un “loop” (s_4, s_4)

cammini

Definizione

Un **cammino** che inizia da s_0 è una sequenza infinita di stati $s_0 s_1 s_2 \dots$ tale che $s_i \rightarrow s_{i+1}$ per ogni $i \in \mathbb{N}$

Intuizione

- un cammino da s_0 rappresenta una **possibile evoluzione** (o computazione) del sistema a partire dallo stato s_0 (non necessariamente lo stato iniziale del sistema)
- consideriamo solo **cammini infiniti** perché, per definizione, la relazione di transizione è totale

Notazione

- scriviamo π^n per l' n -esimo **suffisso** di π
- nota: $\pi^0 = \pi$

bibliografia

Edsger W. Dijkstra. Notes on structured programming. Technical report, Technological University Eindhoven, 1970. URL <https://www.cs.utexas.edu/~EWD/ewd02xx/EWD249.PDF>.